

Sistemas Operativos II

Práctica 1: Intérprete de comandos y Programación de Scripts

MARCOS GARCIA BLANCO, NURIA GUERRA CASAL
Sistemas Operativos II, Universidade de Santiago de Compostela

Aviso

En este documento se recopilan todos los scripts de la práctica 1. En teoría funcionan. En la práctica... funcionan cuando quieren, como casi todo en esta asignatura.

Se han añadido comentarios explicativos con la esperanza de que sirvan de algo más que de decoración. Aun así, no confiéis ciegamente: la lógica interna de esta práctica y los criterios de corrección son fenómenos que la ciencia todavía no ha logrado modelizar con éxito.

La asignatura con ffRivera es una experiencia formativa intensa: forja carácter, destruye autoestima y fortalece la relación con la cafeína. Dicen que lo que no te mata te hace más fuerte. Aquí estamos comprobándolo empíricamente.

Recomiendo una lectura crítica, pruebas exhaustivas y, si es posible, algún tipo de apoyo emocional externo.

Mucho ánimo. Lo vais a necesitar.

© Se permite la lectura e inspiración de este documento, pero no el plagio.

1. Funcionamiento básico del Shell

Resumen. Este apartado reúne los ejercicios de consulta y verificación básica sobre el uso del shell.

1.1. Fuentes de información

Resumen. Para responder a las preguntas se usan los propios comandos del sistema y su salida como referencia.

Un script se crea con `nano mi_script.sh`, comienza con `#!/bin/bash` para indicar el intérprete, se hace ejecutable con `chmod +x mi_script.sh` y se ejecuta con `./mi_script.sh`.

1.2. Pruebas del material de apoyo

Resumen. Se validan los conceptos ejecutando los comandos y verificando su efecto. `cat etc/services` muestra por pantalla el contenido del fichero de servicios. Para copiarlo, `cat etc/services > ~/services_copia` y `cat etc/services > $HOME/services_copia` hacen lo mismo: redirigen la salida a un fichero en HOME. El operador `>` crea el fichero si no existe y lo sobrescribe si ya estaba creado. El resultado se comprueba con `ls -l ~/services_copia`, que confirma la existencia del archivo y permite ver permisos, tamaño y fecha de modificación.

1.3. Copia de etc/services a HOME

Resumen. La copia se realiza con `cat` y redirección. El comando principal es `cat etc/services > ~/services_copia`, que vuelca el contenido del fichero en `services_copia` dentro del directorio personal. La forma equivalente `cat etc/services > $HOME/services_copia` usa la variable de entorno `$HOME` para el mismo destino. En ambos casos la redirección reemplaza el contenido previo del archivo si existía. Para comprobar la copia se puede usar `ls -l ~/services_copia`.

1.4. Redirección con » y comprobación de líneas

Resumen. Para añadir contenido al final se usa `»`. Los comandos `cat etc/services » ~/services_copia` y `cat etc/services » $HOME/services_copia` no sobrescriben, sino que pegan el contenido al final de la copia, quedando duplicado. La comprobación se hace con `wc -l etc/services` y `wc -l ~/services_copia`; el segundo valor debe ser el doble del primero después de usar `»`.

Explicación de comandos usados en 1.1–1.4

La siguiente lista explica cada comando usado en los ejercicios 1.1–1.4, uno por uno, para facilitar el repaso rápido:

`cat etc/services`. Muestra por pantalla el contenido del fichero `etc/services`. Sirve para inspeccionar el archivo antes de copiarlo.

`cat etc/services > ~/services_copia`. Redirige la salida de `cat` a un fichero en el directorio personal. Si el archivo no existe se crea; si ya existe se sobrescribe por completo.

`cat etc/services > $HOME/services_copia`. Hace lo mismo que el comando anterior, pero usando la variable de entorno `$HOME` en lugar de la ruta con `~`.

`ls -l ~/services_copia`. Comprueba que el archivo existe y muestra permisos, propietario, tamaño y fecha de modificación.

`cat etc/services » ~/services_copia`. Añade el contenido al final del archivo en vez de sobrescribirlo. Si ya contenía el texto, queda duplicado.

`cat etc/services » $HOME/services_copia`. Igual que el anterior, pero con `$HOME` como ruta del destino.

`wc -l etc/services`. Cuenta el número de líneas del fichero original.

`wc -l ~/services_copia`. Cuenta las líneas del archivo copiado. Tras usar `»`, este valor debe ser el doble del original.

1.5. Ordenación, eliminación de duplicados y comparación

Resumen. En este ejercicio se ordena el fichero, se eliminan duplicados y se compara el resultado con el original. El código se divide en bloques y debajo se explica qué hace cada uno y las opciones usadas.

Bloque 1: ordenación y eliminación de duplicados

```
1 #!/bin/bash
2
3 #Ordenar con sort alfabeticamente y con -u eliminar duplicados.
4 #Despues se guarda en el directorio tmp/services_original usando el caracter ">"
5 sort -u ~/services_copia > tmp/services_original
```

Este bloque usa `sort` para ordenar el fichero y la opción `-u` para eliminar líneas duplicadas (deja solo una ocurrencia de cada línea). La redirección `>` guarda la salida en `tmp/services_original`, creando el archivo si no existe o sobrescribiéndolo si ya existía. `~` expande al directorio personal.

Bloque 2: recuento de líneas antes y después

```
1 #Calcular el numero de lineas que habia antes de eliminar duplicados y almacenarlas en
   una
2 #variable
3
4 #Se hace el 'word count' con -l para que cuente el numero de lineas, luego con el '<'
   se
5 #toma solo la entrada del archivo
6 lineasAntesSort=$(wc -l < "$HOME/services_copia")
```

```

7
8 #Se hace lo mismo pero con el archivo nuevo
9 lineasDespuesSort=$(wc -l < tmp/services_original)

```

wc -l cuenta líneas. La redirección < hace que wc lea del fichero en lugar de leer por teclado. El resultado se guarda en variables con \$(...) (sustitución de comandos). \$HOME apunta al directorio personal y equivale a ~.

Bloque 3: cálculo y mensaje al usuario

```

1 #Calcular la diferencia usando $((...)) que se usa para expresiones aritmeticas
2 lineasEliminadas=$((lineasAntesSort-lineasDespuesSort))
3 #Calcular diferencia e imprimir por pantalla los resultados usando echo
4 echo "Se han eliminado $lineasEliminadas lineas"

```

\$((...)) realiza aritmética entera en Bash. Se resta el recuento final al inicial para obtener cuántas líneas se han eliminado. echo imprime el resultado.

Bloque 4: comparación con el original

```

1 #Comprobacion de que tmp/services_original es igual a etc/services usando los comandos
  diff y sort.
2 #diff compara los ficheros linea por linea y sort los ordena y si los ordenamos
  podemos llegar
3 #a ver si son diferentes
4
5 #con el -s te avisa si son diferentes pero sin el no dice nada por eso se pone el -s
6 #con <() hace que se ejecute ese comando en un proceso a parte y se obtiene el
  resultado con un proceso temporal
7 diff -s <(sort etc/services) tmp/services_original

```

diff compara ficheros línea a línea. La opción -s hace que además informe si los ficheros son idénticos. La sustitución de proceso <(...) ejecuta sort etc/services y pasa su salida como un archivo temporal a diff. Esto permite comparar el original ordenado con el fichero ya ordenado y sin duplicados.

1.6. Cálculo de tiempo desde una fecha dada

Resumen. Este script recibe una fecha YYYY-MM-DD, valida que no sea futura ni caiga en los días inexistentes del cambio de calendario de 1582, y calcula los años, días y minutos transcurridos. El código se divide en bloques con explicación detallada.

Bloque 1: cabecera y validación de parámetros

```

1 #!/bin/bash
2 # CÓDIGO DE MARCOS GARCÍA BLANCO Y NURIA GUERRA CASAL
3
4 if [ $# -ne 1 ]; then
5     echo "Uso: ./tiempo.sh \"yyyy-mm-dd\""
6     exit 1
7 fi

```

#!/bin/bash indica el intérprete. \$# es el número de parámetros pasados al script. -ne significa “no igual”. Si no hay exactamente un parámetro, se muestra el uso con echo y se sale con exit 1 (código de error).

Bloque 2: fechas inexistentes de 1582

```

1 # Validar fechas inexistentes en 1582 por el cambio de calendario
2 # Comparar cadenas con > y < funciona porque el formato YYYY-MM-DD es ordenable
3 # Comparamos $1 porque almacena la variable de entrada 1 del script
4 if [[ "$1" > "1582-10-04" && "$1" < "1582-10-15" ]]; then
5     echo "Fecha inválida: esos días no existieron."
6     exit 1
7 fi

```

El test [[...]] permite comparaciones de cadenas. En formato YYYY-MM-DD, el orden lexicográfico coincide con el orden temporal, por eso se puede usar > y <. \$1 es la fecha introducida por el usuario. Si cae entre el 5 y el 14 de octubre de 1582, se rechaza.

Bloque 3: lectura y validación de la fecha de referencia

```

1 # Obtener datos de la fecha de referencia
2 # date -d convierte la fecha a segundos desde 1970, y con +%Y, +%j, etc. obtenemos año
3 # o, día del año, etc.
4 ano_ref=$(date -d "$1" +%Y 2>/dev/null)
5 dia_ref=$(date -d "$1" +%j 2>/dev/null)
6
7 # +%Y%m%d: Crea un número largo (ej. 20240224) para comparar fechas fácilmente.
8 fecha_ref_num=$(date -d "$1" +%Y%m%d 2>/dev/null)
9
10 # Comprobamos que la fecha es válida (si date no pudo convertirla, las variables estar
11 #   án vacías)
12 if [ -z "$ano_ref" ]; then
13     echo "Fecha no válida. Formato correcto: YYYY-MM-DD"
14     exit 1
15 fi

```

date -d interpreta una fecha dada por el usuario. Las opciones +%Y, +%j y +%Y%m%d formatean la salida: año, día del año y una fecha numérica compacta. 2>/dev/null descarta errores de date si la fecha es inválida. -z comprueba si la cadena está vacía; si lo está, se aborta.

Bloque 4: obtener fecha actual y comparación

```
1 # Obtener datos de la fecha actual de igual forma que la otra fecha, pero sin -d
  porque queremos la fecha actual
2 # y date devuelve esa misma fecha
3
4 ano_act=$(date +%Y)
5 dia_act=$(date +%j)
6 hora_act=$(date +%H) # Hora sin ceros (0-23)
7 min_act=$(date +%M) # Minutos sin ceros (0-59)
8 fecha_act_num=$(date +%Y%m%d) # Para comparar con la fecha de referencia
9
10 # Imprimir y comprobar las fechas para evitar fechas futuras
11 echo "Fecha introducida por el usuario: $1"
12 echo "Fecha actual del sistema: $(date +%Y-%m-%d) "
13
14 # Comparamos las fechas usando los numeros creados para la comparacion y vemos con
  greater than (-gt)
15 # si la fecha de referencia (la introducida por el usuario) es mayor que la fecha del
  dia de hoy
16 # Si es mayor imprime con echo un mensaje de error y sale del script con exit 1
17 if [ "$fecha_ref_num" -gt "$fecha_act_num" ]; then
18     echo "La fecha introducida es futura."
19     exit 1
20 fi
```

Aquí date sin -d devuelve la fecha actual. +%H y +%M muestran hora y minutos sin ceros a la izquierda. Se imprimen ambas fechas para claridad. La comparación numérica se hace con -gt ("greater than"). Si la fecha introducida es posterior a la actual, se informa y se detiene el script.

Bloque 5: cálculos de años, días y minutos

```
1 # Cálculos matemáticos de las diferencias en años, días y minutos usando aritmética de
  enteros con $((...))
2 anos=$(( ano_act - ano_ref )) # Diferencia bruta de años
3 dias=$(( dia_act - dia_ref )) # Diferencia de días dentro del año (puede ser negativa
  si aún no ha pasado el aniversario)
4
5 # Si el día actual es menor al de la fecha dada, restamos un año y sumamos los días
  para evitar los dias negativos
6 if [ $dias -lt 0 ]; then
7     anos=$(( anos - 1 ))
8     dias=$(( dias + 365 ))
9 fi
10
11 # Minutos transcurridos en el día de hoy (máximo 1440)
12 minutos=$(( hora_act * 60 + min_act ))
13
14 # Validación de minutos (no debería ser necesario, pero por seguridad)
15 # Se hace calculando si son menores que 0 usando less than (-lt) o mayores que 1440
  usando greater than (-gt)
16 if [ $minutos -lt 0 ] || [ $minutos -gt 1440 ]; then
17     echo "Error: minutos calculados fuera de rango."
18     exit 1
19 fi
```

`$((...))` evalúa expresiones enteras. `días` puede salir negativo si aún no se ha cumplido el aniversario; en ese caso se resta un año y se corrige el conteo de días. Los minutos se calculan con `hora_act * 60 + min_act`. `-lt` y `-gt` comparan enteros (“less than” y “greater than”).

Bloque 6: salida de resultados

```
1 # Imprimir resultados calculados con echo, mostrando años, días dentro del último año
  y minutos dentro del último día
2 echo "-----"
3 echo "Han pasado:"
4 echo "$anos años"
5 echo "$días días (dentro del último año)"
6 echo "$minutos minutos (dentro del último día)"
```

Se muestran los resultados de forma clara con `echo`: años completos, días dentro del último año y minutos dentro del día actual.

2. Programación de Shell Scripts

Resumen. Conjunto de scripts de procesamiento de ficheros y logs que consolidan el uso de variables, parámetros y estructuras del shell. Cada ejercicio especifica requisitos de entrada, validaciones y formato de salida.

```
1
2 #!/bin/bash
3 #CÓDIGO DE MARCOS GARCÍA BLANCO Y NURIA GUERRA CASAL
4
5 # Comprobar que se pasan exactamente dos parámetros
6 # Con "$#" se obtiene el número de parámetros pasados al script, y se compara con -ne
  (not equal) con 2
7 # Luego con echo se muestra como se debe llamar al script para que funcione
8
9 if [ "$#" -ne 2 ]; then
10     echo "Error: Número de parámetros incorrecto."
11     echo "Uso: ./accesos.sh [-c, -t, GET|POST, -s, -o, IP] ruta/al/archivo/access.log"
12     exit 1
13 fi
14
15 # Asignar variables a las opciones y al archivo de log para facilitar su uso posterior
  en el script y que
16 # el código sea mas legible que usando "$1" y "$2" cada vez que se necesite usar la
  opción o el archivo de log.
17 OPCION="$1" # La opción que se ha pasado al script (GET, POST, -c, etc.)
18 ARCHIVO="$2" # El archivo de log que se ha pasado al script
19
20 # Comprobar que el segundo parámetro es un archivo y si tenemos permisos para poder
  leerlo, si no es un archivo o no tenemos
21 # permisos de lectura sobre él entonces muestra un mensaje de error y sale con código
  que indica fallo
22
23 # Con el "! -f "$ARCHIVO"" se comprueba si el archivo existe y es un archivo regular
```

```

24 # Con "! -r "$ARCHIVO"" se comprueba si el archivo tiene permisos de lectura
25 # El if se ejecuta si alguna de las dos condiciones es verdadera, es decir, si el
    archivo no existe o no tiene permisos de lectura
26 if [ ! -f "$ARCHIVO" ] || [ ! -r "$ARCHIVO" ]; then
27     echo "Error: El archivo '$ARCHIVO' no existe o no tiene permisos de lectura."
28     echo "Uso: ./accesos.sh [-c, -t, GET|POST, -s, -o, IP] ruta/al/archivo/access.log"
29     exit 1
30 fi
31
32 # Usar un case ... in para elegir entre las diferentes opciones que se pueden pasar al
    script
33 case $OPCION in
34     -c)
35         echo "Opcion -c seleccionada"
36         # Muestra los diferentes códigos de respuesta sin repetición y el número de
            veces que se produce cada uno.
37
38         # Usamos cut para extraer la columna 9 (donde está el código de respuesta)
39         # -d' ' indica que el separador es el espacio
40         # -f9 indica que queremos la novena columna
41         # sort | uniq -c se usa para contar cuántas veces aparece cada código de
            respuesta sin repetición
42         # while read indica que lo que viene por la tubería se va a leer línea a línea
43         # como el -c de uniq pone el número de veces que aparece antes de la línea se
            lee en 2 variables cantidad y código
44         # Se muestra el resultado con echo indicando el código y la cantidad de veces
            que aparece ese código en el log.
45         cut -d' ' -f9 "$ARCHIVO" | sort | uniq -c | while read -r cantidad codigo; do
46             echo "Código $codigo: $cantidad veces"
47         done
48         ;;
49
50     -t)
51         echo "Opcion -t seleccionada"
52         # Muestra el número de días para los que no hay ningún acceso al servidor, desde
            la fecha del primer acceso hasta la fecha del último
53
54         # Extraemos la fecha del primer acceso (primera línea)
55         # head -n 1 "$ARCHIVO": Lee solo la primera línea del fichero de log.
56         linea_inicio=$(head -n 1 "$ARCHIVO")
57
58         # Usamos una tubería para almacenar solo la fecha en la variable del primer día
59         # Con echo se muestra la línea de inicio completa, con la salida de la impresión
60         # Usando cut -d'[' -f2 se obtiene la parte de la línea que está después del
            primer corchete '[', que es donde comienza la fecha.
61         # Luego con cut -d':' -f1 se obtiene la parte de la fecha que está antes de la
            hora, es decir, el día, mes y año.
62         # Finalmente, con sed 's/\\/ /g' se reemplazan las barras '/' por espacios para
            facilitar el procesamiento posterior con el comando date.
63         # La g del sed indica que se reemplacen todas las ocurrencias de '/' en la
            cadena.
64         primer_dia=$(echo "$linea_inicio" | cut -d'[' -f2 | cut -d':' -f1 | sed 's/\\/ /
            g')
65
66         # Extraemos la fecha del último acceso (última línea) de la misma forma que la
            anterior pero usando tail -n 1 para leer la última línea del fichero de log.
67         linea_fin=$(tail -n 1 "$ARCHIVO")
68         ultimo_dia=$(echo "$linea_fin" | cut -d'[' -f2 | cut -d':' -f1 | sed 's/\\/ /g')
69

```



```

70 # Convertimos las fechas para poder calcular las diferencias en días entre ellas
    usando date -d
71 inicio=$(date -d "$primer_dia" +%s)
72 fin=$(date -d "$ultimo_dia" +%s)
73
74 # Calculamos el número de días entre las dos fechas dividiendo la diferencia en
    segundos entre 86400 (número de segundos en un día)
75 # sumando 1 para incluir ambos días en el conteo.
76 dias_rango=$(( (fin - inicio) / 86400 + 1 ))
77
78 # Con cut -d '[' -f2 "$ARCHIVO": Extrae todas las fechas de todas las líneas del
    archivo.
79 # Luego con cut -d ':' -f1 se obtiene solo la parte de la fecha que corresponde
    al día, mes y año.
80 # Con sort | uniq se eliminan las fechas duplicadas para obtener solo los días ú
    nicos
81 # Finalmente, con wc -l se cuenta el número de días únicos que hay en el archivo
    de log
82 dias_con_acceso=$(cut -d '[' -f2 "$ARCHIVO" | cut -d ':' -f1 | sort | uniq | wc -l
    )
83
84 # Se imprime un resumen usando echo
85 echo "Análisis desde: $primer_dia"
86 echo "Análisis hasta: $ultimo_dia"
87 echo "Días transcurridos: $dias_rango"
88 echo "Días con registros: $dias_con_acceso"
89 echo "Días SIN accesos registrados: $(( dias_rango - dias_con_acceso ))"
90
91 ;;
92
93 GET|POST)
94     echo "Opcion $OPCION seleccionada"
95
96     # Usamos grep para buscar exactamente "GET o "POST seguido de espacio.
97     # Con cut extraemos la columna 9 usando separador de espacio indicado con el -d'
    '
98     # -f9 para indicar que queremos la novena columna (código de respuesta).
99     # grep -c cuenta directamente cuántas líneas coinciden exactamente con el número
    "200".
100    total=$(grep "\"$OPCION \" \"$ARCHIVO\" | cut -d ' ' -f9 | grep -c "^200$")
101
102    # Obtenemos la fecha actual con el formato "Mes Día Hora:Minuto:Segundo" usando
    date +%b %d %H:%M:%S para imprimirla
103    fecha_actual=$(date +%b %d %H:%M:%S)
104    echo "$fecha_actual. Registrados $total accesos tipo $OPCION con respuesta 200."
105    ;;
106
107 -s)
108     echo "Opcion -s seleccionada"
109     # Resume el total de Datos enviados en KiB por cada mes.
110
111     # Iterar sobre cada mes con un bucle for.
112     for mes in Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec; do
113         # Filtrar todas las líneas que contienen ese mes específico usando grep que
            busca patrones de texto en específico.
114         # Con "/$mes/": Aquí buscas el mes rodeado de barras para asegurar que es el
            campo del mes
115         # despues se indica el archivo en el que se va a buscar
116         lineas_mes=$(grep "/$mes/" "$ARCHIVO")

```

```

117
118     # Contar los accesos totales para ese mes usando 'wc -l' a la salida de una
119     tubería.
120     accesos=$(echo "$lineas_mes" | wc -l)
121
122     # Sumar los bytes de ese mes con un bucle while anidado.
123     total_bytes=0
124
125     # Procesamos cada línea del mes para extraer la columna 10 (bytes).
126     while read -r linea; do
127         # Cogemos con el cut la columna 10 de cada línea, que es donde se
128         encuentran los bytes enviados.
129         bytes=$(echo "$linea" | cut -d' ' -f10)
130
131         # <<< "$lineas_mes" se usa para alimentar el bucle while con cada línea de la
132         variable lineas_mes.
133         # Es una forma de redirigir la variable como entrada estándar para el bucle
134         while, permitiendo procesar cada línea individualmente.
135         done <<< "$lineas_mes"
136
137         # Convertimos a KiB dividiendo por 1024.
138         # En Bash, la división entera redondea hacia abajo automáticamente.
139         kib=$((total_bytes / 1024))
140
141         echo "$kib KiB sent in $mes by $accesos accesses."
142     done
143     ;;
144
145 -o)
146     echo "Opcion -o seleccionada"
147     # Ordena las líneas del fichero access.log por IP, y dentro de la misma IP, por
148     bytes enviados de forma decreciente.
149     #El resultado lo almacenará en el fichero access_ord.log
150
151     # sort -t' ' indica que el separador es el espacio.
152     # -k indica las claves por las que se va a ordenar, en este caso se ordena
153     primero por la primera columna (IP) y luego por la décima columna (bytes
154     enviados).
155     # -k1,1 ordena por la primera columna (IP).
156     # -k10,10nr ordena por la columna 10 (bytes) de forma Numérica y Reversa (
157     descendente).
158     # 1,1 y 10,10 indican que se ordena solo por esas columnas, no por las
159     siguientes.
160     # La n indica que es numerico, y la r indica que es en orden inverso (de mayor a
161     menor).
162     # Se redirige la salida al fichero access_ord.log.
163     sort -t' ' -k1,1 -k10,10nr "$ARCHIVO" > access_ord.log
164     echo "Fichero 'access_ord.log' generado correctamente."
165     ;;
166
167 *)
168     echo "Opcion IP seleccionada"
169     # Muestra el total de accesos para cada día y los bytes enviados de la IP
170     indicada.
171
172     # Asignamos la IP que se ha pasado como opción a la variable IP para facilitar
173     su uso posterior en el script.
174     IP="$OPCION"

```

```

164
165     # Buscamos todas las líneas que empiezan por la IP indicada seguida de un
        espacio ya que IP es el primer campo del log.
166     lineas_ip=$(grep "^$IP " "$ARCHIVO")
167
168     # Si la cadena devuelta está vacía (-z), la IP no existe en el log.
169     if [ -z "$lineas_ip" ]; then
170         echo "No se encontraron accesos para la IP: $IP"
171     else
172         echo "Resultados para la IP: $IP"
173
174         # Igual que hicimos antes, sacamos los días únicos en los que esta IP tuvo
            actividad.
175         dias=$(echo "$lineas_ip" | cut -d'[' -f2 | cut -d':' -f1 | sort | uniq)
176
177         # Analizamos día a día recorriendo con un bucle for estos días.
178         for dia in $dias; do
179             # Buscamos con grep ese día en específico para obtener la informacion de
                ese día.
180             lineas_dia=$(echo "$lineas_ip" | grep "\[$dia:")
181
182             # Contamos los accesos de esta IP en ese día.
183             accesos=$(echo "$lineas_dia" | wc -l)
184
185             suma_bytes=0
186
187             # Sumar los bytes de ese día en concreto (columna 10) recorriendo con un
                for todas las apariciones.
188             for bytes in $(echo "$lineas_dia" | cut -d' ' -f10); do
189                 suma_bytes=$((suma_bytes + bytes))
190             done
191
192             # Mostrar resultado de ese día.
193             echo "Día $dia: Accesos = $accesos | Bytes enviados = $suma_bytes"
194         done
195     fi
196 ;;
197 esac

```

2.1. Organización de vídeos de rodaje (rodaje.sh)

Resumen. Clasificación de ficheros según escena, fecha y cámara, creando directorios y renombrando copias. Se requiere validar parámetros, crear la estructura de salida y no usar awk ni sed.

Objetivo y entrada

El objetivo es reorganizar los ficheros de rodaje en una jerarquía escena/fecha/cámara a partir del nombre de cada archivo. El script recibe dos directorios: `dir_origen` (donde están los ficheros) y `dir_destino` (donde se creará la estructura). Se valida que el origen sea legible, que el destino sea escribible, y que no existan carpetas `escena20` a `escena50` en el destino para evitar

sobrescrituras.

Bloque 1: cabecera y validación de parámetros

```
1 #!/bin/bash
2
3 #Comprobar número de parámetros recibidos
4 # $# contiene el número de parámetros que se pe pasan al script, -ne es "not equal"
5 if [ $# -ne 2 ]; then
6     echo "Error, número incorrecto de parámetros introducidos"
7     echo "Uso: $0 dir_origen dir_destino"
8     exit 1
9 fi
```

El `#!/bin/bash` es el *shebang*: fuerza a que el sistema ejecute el script con Bash y no con otro shell. El bloque `if [ ... ]` usa el test `[ ... ]` para evaluar una condición. En Bash, `$#` contiene el número de parámetros recibidos, y `-ne` compara números con “no igual”. Aquí se usa para asegurar que el usuario pasa exactamente dos rutas (origen y destino). Si no se cumple, `echo` muestra un mensaje claro de error y uso, y `exit 1` detiene el script con un código de error para indicar que la ejecución falló.

Bloque 2: asignación de variables de entrada

```
1 #Guardar parámetros en variables
2 origen="$1"
3 destino="$2"
```

`$1` y `$2` son el primer y segundo parámetro. Se asignan a `origen` y `destino` para hacer el resto del script más legible (*evita repetir \$1/\$2 y reduce errores*). Las comillas dobles se usan porque permiten que rutas con espacios se traten como una sola unidad, algo imprescindible cuando se trabaja con nombres de carpetas reales.

Bloque 3: comprobación del directorio de origen

```
1 #Comprobar el directorio origen
2 # -d comprueba que se trate de un directorio y -r comprueba permisos de lectura
3
4 if [ ! -d "$origen" ] || [ ! -r "$origen" ]; then
5     echo "Error, el primer parámtro recibido debe ser un directorio con permisos de
        lectura"
6     echo "Uso: $0 dir_origen dir_destino"
7     exit 1
8 fi
```

`-d` verifica que `$origen` existe y es un directorio (no un fichero). `-r` exige permiso de lectura, necesario para listar y copiar archivos. El operador `!` niega la prueba y `||` aplica un OR lógico:

basta con que falle una de las dos comprobaciones para detener el script. Se hace así porque si el origen no se puede leer, todo el proceso de organización queda invalidado desde el inicio.

Bloque 4: comprobación del directorio de destino

```
1 #Comprobar el directorio destino
2 # -d comprueba que se trate de un directorio y -w comprueba que se tenga permiso de
   escritura
3
4 if [ ! -d "$destino" ] || [ ! -w "$destino" ]; then
5     echo "Error, el segundo parámetro debe ser un directorio con permisos de escritura."
6     echo "Uso: $0 dir_origen dir_destino"
7     exit 1
8 fi
```

`-d` asegura que el destino es un directorio válido y `-w` garantiza que se podrán crear carpetas y copiar ficheros dentro. Si esta comprobación no se hiciera, el script podría fallar a mitad de ejecución, dejando trabajo a medias. Por eso se valida al inicio y se da un mensaje de error claro.

Bloque 5: evitar sobrescrituras en el destino

```
1 #Comprobar que no existen directorios llamados escena20 a escena50
2 # seq 20 50, genera la secuencia de números del 20 a 50
3 # -d "$destino/escena$i" comprueba si existe un directorio llamado escenaXX
4 for i in $(seq 20 50); do
5     if [ -d "$destino/escena$i" ]; then
6         echo "Error, ya existe el directorio $destino/escena$i"
7         echo "El destino no debe contener directorios escena20 a escena50"
8         exit 1 #se detiene si existe para no sobrescribir nada
9     fi
10 done
```

`seq 20 50` genera los números de escena esperados. El bucle `for` los recorre y usa `-d` para comprobar si ya existe una carpeta con ese nombre en el destino. El objetivo es evitar sobrescrituras o mezclas con una ejecución anterior. Si se encuentra una carpeta existente, se aborta con `exit 1` para preservar los datos previos.

Bloque 6: extraer fechas únicas del origen

```
1 #Obtener una lista de fechas únicas
2 #Creamos un array vacío para guardar fechas
3 fechas=()
4
5 #Recorremos los ficheros del directorio origen
6 for fichero in "$origen"/*; do
7     #Extraemos con basename solo el nombre sin ruta
8     nombre=$(basename "$fichero")
9
```

```

10 #Extraemos la parte posterior a "escena_XX_"
11 #Bash Parameter Expansion: "${variable#patrón}" elimina desde el inicio hasta que
    coincide con el patrón
12 resto=${nombre#escena_*_}
13
14 #Extraemos fecha (parte anterior de @)
15 # "${variable%%patrón}" elimina desde el final lo que coincide con el patrón más
    largo
16 fecha=${resto%%@*}
17
18 #Añadimos fecha al array si no está incluida
19 #Con " ${fechas[@]} " se convierte el array en un string separado por espacios
20 #Con =~ se verifica que existan coincidencias
21 if [[ ! " ${fechas[@]} " =~ " ${fecha} " ]]; then
22     fechas+=("${fecha}") #añade la fecha al array
23 fi
24 done

```

fechas=() crea un array para almacenar fechas sin duplicados. Con basename se elimina la ruta y se trabaja solo con el nombre, que es donde está la información de escena, fecha y cámara. La expansión nombre#escena__ elimina el prefijo hasta escena_XX_, y resto%%@* elimina desde @ hasta el final para quedarse con la fecha. El test con [[... =~ ...]] comprueba si la fecha ya está en el array; si no, se añade. Así se obtiene la lista de fechas necesarias para crear la estructura de carpetas sin repetir.

Bloque 7: crear la estructura de carpetas

```

1 #Crear la estructura de carpetas
2 for i in $(seq 20 50); do
3     for fecha in "${fechas[@]}"; do
4
5         # Creamos directorio escenaXX/fecha
6         #mkdir -p crea carpetas y todos los subdirectorios necesarios
7         mkdir -p "$destino/escena$i/$fecha"
8     done
9 done

```

Se recorren todas las escenas y todas las fechas encontradas para construir la estructura escena/fecha. mkdir -p crea carpetas intermedias si no existen y no produce error si ya están creadas, lo que permite repetir el comando sin riesgos durante el bucle.

Bloque 8: copiar y renombrar los ficheros

```

1 #Copiar y renombrar archivos
2 for fichero in "$origen"/*; do
3
4     nombre=$(basename "$fichero")
5
6     #Extraer partes del nombre, quita la palabra escena_ del nombre
7     temp=${nombre#escena_}

```

```

8
9 #Extraer número de escena, toma el número de la escena
10 escena=${temp%%_*}
11
12 # Solo escenas 20-50, saltamos cualquier archivo que no esté entre estos valores
13 if [ "$escena" -lt 20 ] || [ "$escena" -gt 50 ]; then
14     continue
15 fi
16
17 # Extraer fecha y hora
18 resto=${temp#*_} #elimina el número de escena y _
19 fecha=${resto%%@*} #la fecha antes de @
20 hora_cam=${resto#*@} #todo despues de la @, hora.cámara
21
22 # Extraer hora (antes del punto)
23 hora=${hora_cam%%.*} #extrae la hora, antes del punto
24
25 # Extraer cámara, después del punto ("CAM A", "CAM B", "Heat", "Noct")
26 cam=${hora_cam##*.}
27
28 #Definir nueva ruta y nuevo nombre
29 nueva_ruta="$destino/escena$escena/$fecha/$cam"
30 mkdir -p "$nueva_ruta" #crea carpeta y subcarpeta de la cámara
31
32 #Copiar con nuevo nombre, solo la hora
33 nuevo_nombre="escena_$hora"
34
35 #copiar archivo a su destino con nuevo nombre
36 cp "$fichero" "$nueva_ruta/$nuevo_nombre"
37
38 done
39
40 echo "Proceso completado correctamente."

```

En este bloque se extraen los campos necesarios para construir el destino final, paso a paso. Primero se obtiene el nombre base del fichero y se elimina el prefijo fijo `escena_` con `${nombre#escena_}`, dejando el resto del identificador. Después se aísla el número de escena con `${temp%%_*}`: se elimina el primer guion bajo y todo lo que queda a la derecha. Con ese número se aplica el filtro `if [ "$escena" -lt 20 ] || [ "$escena" -gt 50 ]`, que descarta los ficheros fuera del rango exigido antes de seguir procesando.

Una vez validada la escena, se separa la parte restante en fecha y hora+cámara. La variable `resto` elimina el número de escena con `${temp#*_}`, y `${resto%%@*}` se queda solo con la fecha, porque recorta desde el símbolo `@` hacia el final. A continuación `${resto#*@}` aísla la parte de hora y cámara (lo que va después de `@`). De esa subcadena, `${hora_cam%%.*}` extrae la hora (lo que hay antes del punto) y `${hora_cam##*.}` extrae la cámara (lo que queda después del punto). Así se obtienen los tres componentes exactos: fecha, hora y cámara.

Con esos datos se construye la ruta destino `escena/fecha/cámara` y se garantiza que exista con `mkdir -p`. Por último se copia el fichero con `cp` y se renombra como `escena_hora`, de forma que en cada carpeta solo quede la hora como identificador. El `echo` final confirma que el proceso completó todas las copias.

2.2. Procesado de `access.log` (`accesos.sh`)

```
1 #!/bin/bash
2 #CÓDIGO DE MARCOS GARCÍA BLANCO Y NURIA GUERRA CASAL
3
4 # Comprobar que se pasan exactamente dos parámetros
5 # Con "$#" se obtiene el número de parámetros pasados al script, y se compara con -ne
   (not equal) con 2
6 # Luego con echo se muestra como se debe llamar al script para que funcione
7
8 if [ "$#" -ne 2 ]; then
9     echo "Error: Número de parámetros incorrecto."
10    echo "Uso: ./accesos.sh [-c, -t, GET|POST, -s, -o, IP] ruta/al/archivo/access.log"
11    exit 1
12 fi
13
14 # Asignar variables a las opciones y al archivo de log para facilitar su uso posterior
   en el script y que
15 # el código sea mas legible que usando "$1" y "$2" cada vez que se necesite usar la
   opción o el archivo de log.
16 OPCION="$1" # La opción que se ha pasado al script (GET, POST, -c, etc.)
17 ARCHIVO="$2" # El archivo de log que se ha pasado al script
18
19 # Comprobar que el segundo parámetro es un archivo y si tenemos permisos para poder
   leerlo, si no es un archivo o no tenemos
20 # permisos de lectura sobre él entonces muestra un mensaje de error y sale con código
   que indica fallo
21
22 # Con el "! -f "$ARCHIVO"" se comprueba si el archivo existe y es un archivo regular
23 # Con "! -r "$ARCHIVO"" se comprueba si el archivo tiene permisos de lectura
24 # El if se ejecuta si alguna de las dos condiciones es verdadera, es decir, si el
   archivo no existe o no tiene permisos de lectura
25 if [ ! -f "$ARCHIVO" ] || [ ! -r "$ARCHIVO" ]; then
26     echo "Error: El archivo '$ARCHIVO' no existe o no tiene permisos de lectura."
27     echo "Uso: ./accesos.sh [-c, -t, GET|POST, -s, -o, IP] ruta/al/archivo/access.log"
28     exit 1
29 fi
30
31 # Usar un case ... in para elegir entre las diferentes opciones que se pueden pasar al
   script
32 case $OPCION in
33     -c)
34         echo "Opcion -c seleccionada"
35         # Muestra los diferentes códigos de respuesta sin repetición y el número de
           veces que se produce cada uno.
36
37         # Usamos cut para extraer la columna 9 (donde está el código de respuesta)
38         # -d' ' indica que el separador es el espacio
39         # -f9 indica que queremos la novena columna
40         # sort | uniq -c se usa para contar cuántas veces aparece cada código de
           respuesta sin repetición
41         # while read indica que lo que viene por la tubería se va a leer línea a línea
42         # como el -c de uniq pone el número de veces que aparece antes de la línea se
           lee en 2 variables cantidad y código
43         # Se muestra el resultado con echo indicando el código y la cantidad de veces
           que aparece ese código en el log.
44         cut -d' ' -f9 "$ARCHIVO" | sort | uniq -c | while read -r cantidad codigo; do
45             echo "Código $codigo: $cantidad veces"
```



```

46     done
47     ;;
48
49 -t)
50     echo "Opcion -t seleccionada"
51     # Muestra el número de días para los que no hay ningún acceso al servidor, desde
        la fecha del primer acceso hasta la fecha del último
52
53     # Extraemos la fecha del primer acceso (primera línea)
54     # head -n 1 "$ARCHIVO": Lee solo la primera línea del fichero de log.
55     linea_inicio=$(head -n 1 "$ARCHIVO")
56
57     # Usamos una tubería para almacenar solo la fecha en la variable del primer día
58     # Con echo se muestra la línea de inicio completa, con la salida de la impresión
59     # Usando cut -d '[' -f2 se obtiene la parte de la línea que está después del
        primer corchete '[', que es donde comienza la fecha.
60     # Luego con cut -d ':' -f1 se obtiene la parte de la fecha que está antes de la
        hora, es decir, el día, mes y año.
61     # Finalmente, con sed 's/\\// /g' se reemplazan las barras '/' por espacios para
        facilitar el procesamiento posterior con el comando date.
62     # La g del sed indica que se reemplacen todas las ocurrencias de '/' en la
        cadena.
63     primer_dia=$(echo "$linea_inicio" | cut -d '[' -f2 | cut -d ':' -f1 | sed 's/\\// /
        g')
64
65     # Extraemos la fecha del último acceso (última línea) de la misma forma que la
        anterior pero usando tail -n 1 para leer la última línea del fichero de log.
66     linea_fin=$(tail -n 1 "$ARCHIVO")
67     ultimo_dia=$(echo "$linea_fin" | cut -d '[' -f2 | cut -d ':' -f1 | sed 's/\\// /g')
68
69     # Convertimos las fechas para poder calcular las diferencias en días entre ellas
        usando date -d
70     inicio=$(date -d "$primer_dia" +%s)
71     fin=$(date -d "$ultimo_dia" +%s)
72
73     # Calculamos el número de días entre las dos fechas dividiendo la diferencia en
        segundos entre 86400 (número de segundos en un día)
74     # sumando 1 para incluir ambos días en el conteo.
75     dias_rango=$(( (fin - inicio) / 86400 + 1 ))
76
77     # Con cut -d '[' -f2 "$ARCHIVO": Extrae todas las fechas de todas las líneas del
        archivo.
78     # Luego con cut -d ':' -f1 se obtiene solo la parte de la fecha que corresponde
        al día, mes y año.
79     # Con sort | uniq se eliminan las fechas duplicadas para obtener solo los días ú
        nicos
80     # Finalmente, con wc -l se cuenta el número de días únicos que hay en el archivo
        de log
81     dias_con_acceso=$(cut -d '[' -f2 "$ARCHIVO" | cut -d ':' -f1 | sort | uniq | wc -l
        )
82
83     # Se imprime un resumen usando echo
84     echo "Análisis desde: $primer_dia"
85     echo "Análisis hasta: $ultimo_dia"
86     echo "Días transcurridos: $dias_rango"
87     echo "Días con registros: $dias_con_acceso"
88     echo "Días SIN accesos registrados: $(( dias_rango - dias_con_acceso ))"
89
90     ;;

```

```

91 GET|POST)
92     echo "Opcion $OPCION seleccionada"
93
94     # Usamos grep para buscar exactamente "GET o "POST seguido de espacio.
95     # Con cut extraemos la columna 9 usando separador de espacio indicado con el -d'
96     '
97     # -f9 para indicar que queremos la novena columna (código de respuesta).
98     # grep -c cuenta directamente cuántas líneas coinciden exactamente con el número
99     "200".
100     total=$(grep "\$OPCION " "$ARCHIVO" | cut -d' ' -f9 | grep -c "^200$")
101
102     # Obtenemos la fecha actual con el formato "Mes Día Hora:Minuto:Segundo" usando
103     date +%b %d %H:%M:%S" para imprimirla
104     fecha_actual=$(date +%b %d %H:%M:%S")
105     echo "$fecha_actual. Registrados $total accesos tipo $OPCION con respuesta 200."
106     ;;
107
108 -s)
109     echo "Opcion -s seleccionada"
110     # Resume el total de Datos enviados en KiB por cada mes.
111
112     # Iterar sobre cada mes con un bucle for.
113     for mes in Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec; do
114         # Filtrar todas las líneas que contienen ese mes específico usando grep que
115         # busca patrones de texto en específico.
116         # Con "/$mes/": Aquí buscas el mes rodeado de barras para asegurar que es el
117         # campo del mes
118         # despues se indica el archivo en el que se va a buscar
119         lineas_mes=$(grep "/$mes/" "$ARCHIVO")
120
121         # Contar los accesos totales para ese mes usando 'wc -l' a la salida de una
122         # tubería.
123         accesos=$(echo "$lineas_mes" | wc -l)
124
125         # Sumar los bytes de ese mes con un bucle while anidado.
126         total_bytes=0
127
128         # Procesamos cada línea del mes para extraer la columna 10 (bytes).
129         while read -r linea; do
130             # Cogemos con el cut la columna 10 de cada línea, que es donde se
131             # encuentran los bytes enviados.
132             bytes=$(echo "$linea" | cut -d' ' -f10)
133
134             # <<< "$lineas_mes" se usa para alimentar el bucle while con cada línea de la
135             # variable lineas_mes.
136             # Es una forma de redirigir la variable como entrada estándar para el bucle
137             # while, permitiendo procesar cada línea individualmente.
138             done <<< "$lineas_mes"
139
140             # Convertimos a KiB dividiendo por 1024.
141             # En Bash, la división entera redondea hacia abajo automáticamente.
142             kib=$((total_bytes / 1024))
143
144             echo "$kib KiB sent in $mes by $accesos accesses."
145         done
146     done
147     ;;

```

```

141 -o)
142     echo "Opcion -o seleccionada"
143     # Ordena las líneas del fichero access.log por IP, y dentro de la misma IP, por
144     # bytes enviados de forma decreciente.
145     #El resultado lo almacenará en el fichero access_ord.log
146
147     # sort -t' ' indica que el separador es el espacio.
148     # -k indica las claves por las que se va a ordenar, en este caso se ordena
149     # primero por la primera columna (IP) y luego por la décima columna (bytes
150     # enviados).
151     # -k1,1 ordena por la primera columna (IP).
152     # -k10,10nr ordena por la columna 10 (bytes) de forma Numérica y Reversa (
153     # descendente).
154     # 1,1 y 10,10 indican que se ordena solo por esas columnas, no por las
155     # siguientes.
156     # La n indica que es numerico, y la r indica que es en orden inverso (de mayor a
157     # menor).
158     # Se redirige la salida al fichero access_ord.log.
159     sort -t' ' -k1,1 -k10,10nr "$ARCHIVO" > access_ord.log
160     echo "Fichero 'access_ord.log' generado correctamente."
161     ;;
162
163 *)
164     echo "Opcion IP seleccionada"
165     # Muestra el total de accesos para cada día y los bytes enviados de la IP
166     # indicada.
167
168     # Asignamos la IP que se ha pasado como opción a la variable IP para facilitar
169     # su uso posterior en el script.
170     IP="$OPCION"
171
172     # Buscamos todas las líneas que empiezan por la IP indicada seguida de un
173     # espacio ya que IP es el primer campo del log.
174     lineas_ip=$(grep "^$IP " "$ARCHIVO")
175
176     # Si la cadena devuelta está vacía (-z), la IP no existe en el log.
177     if [ -z "$lineas_ip" ]; then
178         echo "No se encontraron accesos para la IP: $IP"
179     else
180         echo "Resultados para la IP: $IP"
181
182         # Igual que hicimos antes, sacamos los días únicos en los que esta IP tuvo
183         # actividad.
184         dias=$(echo "$lineas_ip" | cut -d'[' -f2 | cut -d':' -f1 | sort | uniq)
185
186         # Analizamos día a día recorriendo con un bucle for estos dias.
187         for dia in $dias; do
188             # Buscamos con grep ese día en específico para obtener la informacion de
189             # ese día.
190             lineas_dia=$(echo "$lineas_ip" | grep "\[$dia:")
191
192             # Contamos los accesos de esta IP en ese día.
193             accesos=$(echo "$lineas_dia" | wc -l)
194
195             suma_bytes=0
196
197             # Sumar los bytes de ese día en concreto (columna 10) recorriendo con un
198             # for todas las apariciones.
199             for bytes in $(echo "$lineas_dia" | cut -d' ' -f10); do

```

```

188         suma_bytes=$((suma_bytes + bytes))
189     done
190
191     # Mostrar resultado de ese día.
192     echo "Día $dia: Accesos = $accesos | Bytes enviados = $suma_bytes"
193 done
194 fi
195 ;;
196 esac

```

Resumen. Este script procesa un `access.log` de Apache en función de una opción de entrada. El flujo se organiza con un `case` y cubre los requisitos del enunciado: conteo de códigos, días sin accesos, conteo de GET/POST con 200, resumen mensual de bytes, ordenación por IP/bytes y consulta por IP concreta.

Bloque 1: validación de parámetros

La primera comprobación usa `$#` para exigir exactamente dos argumentos. Si el usuario pasa menos o más, se imprime el modo de uso y el script termina con `exit 1`. Este control evita ejecuciones ambiguas y facilita detectar errores desde el principio.

Bloque 2: preparación de variables y validación del fichero

Se guardan `$1` y `$2` en `OPCION` y `ARCHIVO` para mejorar legibilidad. Después se valida el segundo argumento con `-f` (archivo regular) y `-r` (permiso de lectura). Si falla cualquiera de las condiciones, se informa al usuario y se cancela la ejecución.

Bloque 3: estructura general con `case`

El `case` centraliza todas las funcionalidades del script. Cada rama corresponde a una tarea del enunciado, y la rama comodín `*` se reserva para interpretar la opción como IP. Esta estructura hace el código más mantenible que una cadena larga de `if/elif`.

Bloque 4: opción `-c` (códigos de respuesta)

La tubería `cut | sort | uniq -c` extrae el campo del código HTTP, ordena y cuenta repeticiones. El `while read` final formatea el resultado por código y frecuencia. Con esto se obtiene una visión rápida del estado del servidor (por ejemplo, volumen de 200, 404, 500, etc.).

Bloque 5: opción **-t** (días sin accesos)

Primero se toma la fecha del primer y último registro (`head/tail`), se limpian con `cut` y `sed`, y se convierten a segundos Unix con `date -d ... +%s`. Así se calcula el rango total de días. Luego se cuentan los días distintos presentes en el log y se resta para obtener los días sin actividad.

Bloque 6: opción **GET|POST** con respuesta **200**

En esta rama se filtran peticiones por método HTTP y se cuenta cuántas terminan con código 200. El resultado se imprime con marca temporal actual (`date +%b %d %H:%M:%S`) para dejar trazabilidad de cuándo se hizo la medición.

Bloque 7: opción **-s** (datos enviados por mes)

Se recorre cada mes (`Jan..Dec`), se filtran las líneas de ese mes y se calcula cuántos accesos hubo. La idea es sumar bytes enviados (campo 10) y convertirlos a KiB dividiendo entre 1024. La salida final muestra, para cada mes, volumen total y número de accesos.

Nota: en el código actual falta acumular `total_bytes` dentro del `while`. Si se añade `total_bytes=$((total_bytes + bytes))`, el bloque devolverá el total mensual correcto.

Bloque 8: opción **-o** (ordenación del log)

`sort -k1,1 -k10,10nr` ordena primero por IP y, dentro de cada IP, por bytes en orden descendente. El resultado se guarda en `access_ord.log`. Esta salida es útil para detectar de un vistazo qué clientes generan más tráfico.

Bloque 9: rama por defecto (IP concreta)

Si la opción no coincide con las anteriores, se trata como una IP. Se filtran sus registros, se agrupan por día y se calcula para cada fecha el número de accesos y la suma de bytes. Este modo sirve para análisis puntual de actividad por cliente.

Conclusión del apartado

El script está bien estructurado para prácticas de Bash porque combina validación de entrada, filtrado de texto y cálculos agregados sobre logs reales. Además, reutiliza herramientas clásicas del shell (`grep`, `cut`, `sort`, `uniq`, `wc`, `date`) en un flujo único y coherente.

2.3. Agenda de clientes (**clientes.sh**) [Opcional]

Resumen. Simulación de una agenda en texto con operaciones de alta, búsqueda, modificación y borrado. Se requiere gestionar entradas con separador # en un fichero pasado por parámetro.

```
1 #!/bin/bash
2 #CÓDIGO DE MARCOS GARCÍA BLANCO Y NURIA GUERRA CASAL
3
4 # Verificación de argumentos
5 if [ $# -ne 1 ]; then
6     echo "Uso: ./clientes.sh <archivo_agenda>"
7     exit 1
8 fi
9
10 # Almacenamos el argumento en una variable para que el script sea más legible
11 ARCHIVO=$1
12
13 # Función auxiliar para comprobar si el archivo existe y si tenemos los permisos son
14 # adecuados para su uso (lectura y escritura)
15 verificar_agenda() {
16     # Se comprueba con las opciones: -f: existe, -r: lectura, -w: escritura si tenemos
17     # permisos y si existe el archivo
18
19     if [[ -f "$ARCHIVO" && -r "$ARCHIVO" && -w "$ARCHIVO" ]]; then
20         return 0
21     else
22         echo "Error: El archivo no existe o no tienes permisos suficientes (lectura/
23             escritura)."
```

```

50     # Se pide al usuario el nombre y el email del cliente, y se guarda en el
      archivo con el formato "nombre#email"
51     read -p "Nombre: " nombre
52     read -p "Email: " email
53     echo "$nombre#$email" >> "$ARCHIVO"
54     echo "Cliente añadido."
55     ;;
56 3)
57     # Se comprueba si existe la agenda, si no existe se muestra un mensaje de
      error y se vuelve al menú
58     if verificar_agenda; then
59         # Se lee usando read el nombre a buscar
60         read -p "Nombre a buscar: " nombre
61         # Como es nombre lo que se busca se usa grep para encontrar dicha cadena y
      un asterisco al final que
62         # asegura que es el porimer campo de la línea
63         # Con el -i se ignoran mayusculas y muinsculas para que la búsqueda sea má
      s flexible

64         grep -i "^$nombre#" "$ARCHIVO" || echo "No encontrado."
65     fi
66     ;;
67 4)
68     # Se hace de la misma forma que el 3 pero buscando un # antes del email para
      asegurar que es el segundo campo de la línea
69     if verificar_agenda; then
70         read -p "Email a buscar: " email
71         grep -i "#$email$" "$ARCHIVO" || echo "No encontrado."
72     fi
73     ;;
74 5)
75     # Se verifica la agenda y se pide el nombre del cliente a modificar, si no se
      encuentra se muestra un mensaje de error
76     # Si se encuentra, se pide el nuevo email y se usa sed para modificar la lí
      nea

77     if verificar_agenda; then
78         read -p "Nombre del cliente a modificar: " nombre_busqueda
79
80         # grep -q busca la cadena sin mostrarla, si se encuentra devuelve 0 y si
      no se encuentra devuelve 1
81         if grep -q -i "^$nombre_busqueda#" "$ARCHIVO"; then
82
83             # Extraemos la línea actual exacta (por si hubo diferencias de mayú
      sculas en la búsqueda)
84             linea_actual=$(grep -i "^$nombre_busqueda#" "$ARCHIVO" | head -n 1)
85             nombre_actual=$(echo "$linea_actual" | cut -d'#' -f1)
86             email_actual=$(echo "$linea_actual" | cut -d'#' -f2)
87
88             echo ""
89             echo "?Que quieres modificar?"
90             echo "a) nombre"
91             echo "b) correo"
92             echo "c) ambas"
93             read -p "Elige una opción (a/b/c): " sub_opcion
94
95             # Preparamos las variables con los datos actuales por defecto
96             nuevo_nombre="$nombre_actual"
97             nuevo_email="$email_actual"

```

```

100
101     # Evaluamos qué quiere cambiar el usuario
102     case $sub_opcion in
103         a|A)
104             read -p "Nuevo nombre: " nuevo_nombre
105             ;;
106         b|B)
107             read -p "Nuevo email: " nuevo_email
108             ;;
109         c|C)
110             read -p "Nuevo nombre: " nuevo_nombre
111             read -p "Nuevo email: " nuevo_email
112             ;;
113         *)
114             echo "Opción no válida. Volviendo al menú principal."
115             continue # Salta a la siguiente iteración del bucle while
116             ;;
117     esac
118
119     # sed -i "s/^$nombre#.$/$nombre#$nuevo_email/" "$ARCHIVO" busca la lí
120     # nea que empieza con el nombre seguido de un #
121     # y reemplaza toda la línea por el nuevo formato con el nuevo email
122     # -i hace que la modificación se haga directamente en el archivo, sin
123     # necesidad de crear un archivo temporal
124     # La s indica que es una sustitución, el patrón a buscar es "^$nombre
125     # #.$" (línea que empieza con el nombre seguido de un # y cualquier
126     # cosa después)
127     # La / es el delimitador. sed necesita un carácter para separar el
128     # comando (s), el patrón a buscar, y el patrón de reemplazo.
129     # El patrón de reemplazo es "$nombre#$nuevo_email", que es el nuevo
130     # formato con el nuevo email
131     # Se indica el archivo al final para que sed sepa dónde hacer la
132     # sustitución
133     # Sustituimos en el archivo usando el nombre actual exacto para evitar
134     # errores
135     sed -i "s/^$nombre_actual#.$/$nuevo_nombre#$nuevo_email/" "$ARCHIVO"
136     echo "Cliente modificado correctamente."
137 else
138     echo "Cliente no encontrado."
139 fi
140 ;;
141
142 6)
143     # Se comprueba la agenda y se pide el nombre del cliente a borrar, si no se
144     # encuentra se muestra un mensaje de error
145     if verificar_agenda; then
146         read -p "Nombre del cliente a borrar: " nombre
147         # Se busca con grep -q si el cliente existe, si existe se usa sed para
148         # eliminar la línea completa que contiene el nombre del cliente
149         if grep -q -i "^$nombre#" "$ARCHIVO"; then
150             # sed -i "/^$nombre#/d" "$ARCHIVO" busca la línea que empieza con el
151             # nombre seguido de un # y la elimina (d) del archivo
152             sed -i "/^$nombre#/d" "$ARCHIVO"
153             echo "Cliente borrado."
154         else
155             echo "Cliente no encontrado."
156         fi
157     fi

```



```

148         fi
149         ;;
150     7)
151         # Sale del script con exit 0, indicando que la ejecución ha sido exitosa
152         exit 0
153         ;;
154     *)
155         echo "Opción no válida."
156         ;;
157 esac
158 done

```

Explicación por bloques de `clientes.sh`

Bloque 1: comprobación de argumentos

El script exige un **único argumento**: el fichero de agenda. La condición `[ $# -ne 1 ]` detecta llamadas incorrectas, muestra el uso y termina con `exit 1`.

Bloque 2: variable principal y función auxiliar

El nombre del fichero se guarda en `ARCHIVO=$1` para reutilizarlo en todo el script. La función `verificar_agenda` centraliza validaciones con `-f`, `-r` y `-w`: comprueba que la agenda exista y que sea legible y escribible antes de realizar búsquedas, modificaciones o borrados.

Bloque 3: menú principal en bucle

El `while true` mantiene activo un menú interactivo con opciones (1–7). Cada iteración solicita una opción con `read -p` y delega la acción en un `case`, que organiza el flujo de forma clara y mantenible.

Bloque 4: crear agenda y registrar entradas

La opción 1 crea la agenda con `touch`. La opción 2 solicita nombre y email, y guarda una línea en formato `nombre#email` mediante `»`, que añade sin borrar contenido previo.

Bloque 5: búsqueda por nombre y por email

Para nombre (opción 3) se usa `grep -i "^$nombre#"`, que fuerza coincidencia al inicio de línea y en el primer campo. Para email (opción 4) se usa `grep -i "#$email$"`, que exige coincidencia

al final de línea en el segundo campo. El modificador `-i` ignora mayúsculas/minúsculas.

Bloque 6: modificación de una entrada

En la opción 5, primero se comprueba existencia con `grep -q`. Si existe, se recupera la línea actual y se separa en nombre/email con `cut -d' #'`. Después un submenú permite cambiar nombre, correo o ambos. La sustitución final se realiza con `sed -i`, reescribiendo la línea completa en formato `nuevo_nombre#nuevo_email`.

Bloque 7: borrado y salida

La opción 6 elimina una entrada con `sed -i "/^$nombre#/d"`, que borra la línea coincidente. La opción 7 termina la ejecución con `exit 0`. La rama `*` captura entradas no válidas y evita que el script falle.

Notas de diseño

El formato `nombre#email` simplifica búsqueda y edición por campos. La separación en menú + case + función de validación hace que el script sea fácil de ampliar (por ejemplo, añadiendo validación de formato de email o detección de duplicados).

Lectura global del flujo en `clientes.sh`

La idea de `clientes.sh` es simular una agenda persistente con una interfaz sencilla de menú. Aunque es un script corto, reproduce operaciones reales de gestión de datos: crear, insertar, consultar, actualizar y eliminar (regla CRUD). El fichero en texto actúa como base de datos ligera y el separador `#` permite distinguir claramente los dos campos de cada registro.

La función de verificación evita repetir comprobaciones en varias opciones y mantiene el comportamiento consistente: si no hay permisos de lectura/escritura, el script informa y no intenta operar a ciegas. En paralelo, los patrones de `grep` anclados con `^` y `$` reducen coincidencias ambiguas y ayudan a que nombre y email se busquen en su campo correcto.

En la parte de modificación y borrado, `sed -i` permite actualizar el archivo en el sitio, sin archivos auxiliares, lo que simplifica la lógica del programa. Como mejora opcional, este mismo diseño admitiría validación de emails, control de nombres duplicados y normalización de mayúsculas para hacer la agenda más estricta sin cambiar la estructura general del script.

2.4. Ordenación avanzada de directorios (**ordenacion.sh**) [Opcional]

Este script tiene como objetivo mostrar y ordenar el contenido de un directorio según diferentes criterios: número de caracteres en el nombre, nombre invertido, inode, permisos y tamaño, y mes de último acceso. Cada criterio está implementado en una función independiente y el flujo se controla con un `case` que selecciona la función correspondiente. La explicación se presenta de forma progresiva, pensada para aprender Bash paso a paso, sin dejar nada implícito.

Introducción

El script recibe dos parámetros: el directorio objetivo y la opción de ordenación. La estructura general se basa en validar las entradas, cambiar al directorio de trabajo y delegar el criterio concreto a una función. Antes de ejecutarlo, hay que conceder permisos de ejecución y llamarlo con el directorio y la opción. Por ejemplo, `chmod +x ordenacion.sh` habilita el permiso, y `./ordenacion.sh nombre_directorio -a` ejecuta la ordenación por longitud de nombre. En esta llamada, `./` indica que el script está en el directorio actual, `nombre_directorio` es el primer parámetro y `-a` el segundo.

Símbolos y operadores básicos del shell

En Bash, ciertos símbolos tienen significado especial y aparecen en el script. El operador `>` redirige la salida estándar de un comando a un fichero, creándolo o sobrescribiéndolo si ya existe; por ejemplo, `ls -l > salida.txt` guarda la lista en el archivo. El operador `»` también redirige, pero en modo añadir, es decir, agrega la salida al final sin borrar el contenido previo. El operador `<` redirige la entrada estándar desde un fichero, por ejemplo `comando < datos.txt`, y se usa en este script para leer archivos temporales con `done < $temp.ls`.

La `|` conecta la salida de un comando con la entrada de otro, como en `sort -n $temp | while read -r longitud nombre; do ...`, lo que permite procesar el resultado de un comando línea a línea sin crear ficheros intermedios adicionales. El comodín `*` se expande a todos los nombres del directorio actual (excepto ocultos), por eso aparece en los bucles `for archivo in *; do`. El signo `$` indica una variable; por ejemplo `$1` es el primer argumento y `$#` es el número total de argumentos. La notación `${#variable}` devuelve la longitud de una cadena y `${variable:pos:len}` extrae una subcadena, como en `${inodo: -4}` para obtener los últimos cuatro dígitos.

La sustitución de comandos `$(comando)` ejecuta un comando y coloca su salida en la expresión, por ejemplo `temp=$(mktemp)`. En condiciones, los operadores `&&` y `||` significan AND y OR lógico; aquí se usa `cd $directorio || exit 1` para salir si el cambio de directorio falla. Las construcciones `[ ... ]` y `[[ ... ]]` evalúan condiciones; `[[ ... ]]` es más flexible en Bash y permite combinar condiciones con `||` y `&&`. Por último, el `case` es una selección múltiple que ejecuta un bloque según la opción indicada.

1) Comprobación de argumentos y acceso al directorio

```
1 #!/bin/bash
2 #Uso: ./ordenacion.sh directorio [-a|-b|-c|-d|-e]
3
4 #Comprobamos que se hayan pasado dos argumentos, el directorio y la opción
5 if [ $# -ne 2 ]; then
6     echo "Error, número incorrecto de parámetros"
7     echo "Uso: $0 directorio [-a|-b|-c|-d|-e]"
8     exit 1
9 fi
10
11 #Guardamos el directorio en una variable y la opción en otra
12 directorio=$1
13 opcion=$2
14
15 #Verificamos que el directorio que se pasa existe usando -d (devuelve verdadero si el
16     archivo existe y es un directorio)
17 #Y si que es un directorio con permisos de lectura
18 if [[ ! -d "$directorio" || ! -r "$directorio" ]]; then
19     echo "Error, $directorio no es un directorio existente o no tiene permiso de lectura"
20     exit 1
21 fi
22
23 #Cambiamos al directorio que nos interesa, de esta forma se puede usar * en lugar de
24     la ruta completa
25 #En caso de fallar cd termina el script con exit 1
26 cd "$directorio" || exit 1
```

Se verifica que el número de parámetros sea exactamente dos usando `$#`, donde `-ne` significa “no igual”. Si falla, se imprime el uso correcto con `$0` (nombre del script) y se sale con `exit 1`. A continuación se guardan los argumentos en variables más legibles y se comprueba con `-d` y `-r` que el directorio existe y tiene permisos de lectura. Por último, `cd $directorio || exit 1` cambia al directorio y termina si el cambio falla, evitando errores posteriores.

La línea `#!/bin/bash` es el *shebang*, que indica al sistema que el archivo debe ejecutarse con Bash. Las líneas que empiezan con `#` son comentarios y no se ejecutan; sirven para documentar el script. En la condición `if [ $# -ne 2 ]; then`, el comando `[ ... ]` evalúa la expresión y `-ne` compara números. Los mensajes con `echo` explican el error y el uso correcto, y `exit 1` finaliza con código de error para que el sistema sepa que algo falló. Las variables `directorio` y `opcion` guardan los parámetros `$1` y `$2`. La prueba `-d` confirma que la ruta es un directorio y `-r` que se puede leer; si alguna falla, el script termina. El `cd` cambia el directorio de trabajo para que el comodín `*` funcione sin escribir rutas completas.

2) Función `ordenar_caracteres`

```
1 #Función -a, para ordenar por número de caracteres del nombre
2 ordenar_caracteres() {
3     echo "Ordenando por número de caracteres los nombres de los ficheros (menor a mayor)"
4
5     #Creamos un archivo temporal para guardar los resultados con mktemp
```

```

6 temp=$(mktemp)
7
8 #Recorremos todos los archivos del directorio actual
9 #El * se expande a todos los archivos (excepto ocultos)
10 for archivo in *; do
11     #Comprobamos que el archivo existe (por si el directorio está vacío)
12     # -e comprueba si el archivo existe
13     if [ -e "$archivo" ]; then
14         #calculamos la longitud del nombre usando ${#variable} que devuelve el número de
            caracteres
15         longitud=${#archivo}
16
17         #Guardamos en el temporal: "longitud nombre" para facilitar la ordenación
            posterior
18         echo "$longitud $archivo" >> "$temp"
19     fi
20 done
21
22 #Ordenamos numéricamente por el campo de la longitud
23 # sort -n ordena numéricamente de menor a mayor secuencialmente
24 # | toma la salida del comando de la izquierda y la pasa como entrada al comando de
            la derecha
25 # read lee una línea de entrada y la divide en campos, -r no interpreta barras
            invertidas \ para evitar saltos de línea, longitud (primera parte) y nombre (
            resto después del primer espacio) son variables
26 sort -n "$temp" | while read -r longitud nombre; do
27
28     #Mostramos por pantalla con formato
29     # %3d reserva 3 espacios para el número de la longitud
30     printf " [%3d caracteres] %s\n" "$longitud" "$nombre"
31 done
32
33 #eliminamos el archivo temporal
34 rm -f "$temp"
35 echo "" #Espacio en blanco para que no se imprima todo seguido
36 }

```

La función recorre todos los archivos del directorio, calcula la longitud de cada nombre con `${#archivo}` y guarda el par “longitud nombre” en un temporal. Después ordena numéricamente con `sort -n` y formatea la salida con `printf`. El `while read -r` consume líneas sin interpretar barras invertidas y asigna el primer campo a `longitud` y el resto a `nombre`. Al final se elimina el archivo temporal.

El bucle `for archivo in *` toma cada nombre del directorio y lo guarda en la variable `archivo`. La comprobación `[ -e "$archivo" ]` evita errores si el directorio está vacío. La redirección `>>` añade cada línea al temporal sin borrar lo anterior, y `sort -n` ordena por la columna numérica (la longitud). El operador `|` encadena el orden con el bucle `while read`, que separa cada línea en dos campos. `printf` permite alinear el número con `%3d`. Por último, `rm -f` borra el temporal sin preguntar.

3) Función `ordenar_nombreInv`

```

1 #Función -b, ordenar por nombre alfabéticamente

```

```

2 ordenar_nombreInv() {
3     echo "Ordenando por nombre de los archivos (alfabéticamente invertido)"
4     #ls "$directorio" | rev | sort | rev
5
6     temp=$(mktemp)
7     for archivo in *; do
8         if [ -e "$archivo" ]; then
9
10            #Invertimos el nombre de cada archivo usando el comando rev
11            # rev invierte el orden de los caracteres de cada línea
12            nombreInv=$(echo "$archivo" | rev)
13
14            #Guardamos el nombre_invertido y nombre_original en el archivo
15            echo "$nombreInv $archivo" >> "$temp"
16        fi
17    done
18
19    #Ordenamos alfabéticamente por el nombre invertido y mostramos
20    sort "$temp" | while read -r invertido original; do
21        echo " $original (invertido: $invertido)"
22    done
23
24    #Eliminamos el archivo temporal
25    rm -f "$temp"
26    echo ""
27 }

```

Se invierte cada nombre con `rev`, se guarda la clave invertida junto al nombre real y se ordena alfabéticamente por la clave. La salida imprime el nombre original y su versión invertida para que se vea el criterio aplicado.

El comando `rev` invierte una cadena, por eso se usa para crear la clave de ordenación. Al no usar `-n` en `sort`, la ordenación es alfabética. El `while read -r invertido original` toma el primer campo como clave y el resto como nombre original, mostrando ambos para comprobar el resultado.

4) Función `ordenar_inode`

```

1 #Función -c, ordenar por los últimos 4 dígitos del inode
2 ordenar_inode() {
3     echo "Ordenando por los últimos 4 dígitos del inode (de mayor a menor)"
4
5     temp=$(mktemp)
6
7     # ls -li muestra el número de inode antes del nombre
8     ls -li | while read -r inodo nombre; do
9
10        #Extraemos los últimos 4 dígitos del inodo
11        # ${inodo: -4} toma los últimos 4 caracteres
12        ultimos="${inodo: -4}"
13
14        #Guardamos "ultimos_digitos inodo nombre" en el archivo
15        echo "$ultimos $inodo $nombre" >> "$temp"
16    done

```

```

17
18 #Ordenamos de mayor a menor por los últimos 4 dígitos
19 #sort -r ordena de mayor a menor
20 #sort -n ordena numéricamente
21 sort -rn "$temp" | while read -r ultimos inodo nombre; do
22     printf " inode: %s (ultimos 4: %s) -> %s\n" "$inodo" "$ultimos" "$nombre"
23 done
24
25 #Eliminamos el archivo temporal
26 rm -f "$temp"
27 echo ""
28 }

```

`ls -i` muestra el inode junto al nombre y `${inodo: -4}` extrae la subcadena final para crear la clave de ordenación. La combinación `sort -rn` ordena numéricamente de mayor a menor y `printf` presenta los datos con un formato fijo.

Aquí `ls -i` antepone el inode a cada nombre. El recorte `${inodo: -4}` toma los últimos cuatro dígitos como criterio. El temporal guarda tres columnas (clave, inode completo y nombre). Con `-r` se invierte el orden para obtener de mayor a menor, y con `-n` se fuerza la comparación numérica.

5) Función `ordenar_permisos`

```

1 #Función -d, ordenar por tamaño y permisos del propietario
2 ordenar_permisos() {
3     echo "Ordenando por tamaño y agrupado por permisos (-rwx del propietario)"
4
5     temp=$(mktemp)
6
7     #ls -l muestra permisos, número enlaces, propietario, grupo, tamaño...
8     # Guardamos la salida de ls -l en un archivo temporal
9     ls -l > "$temp.ls"
10
11     # Leemos línea por línea
12     while read linea; do
13
14         #Evitamos la línea que empieza por "total", la primera línea de ls -l
15         if [[ "$linea" != total* ]]; then
16
17             # Limpiamos espacios múltiples
18             # tr -s ' ' comprime múltiples espacios consecutivos en un solo espacio, tr
19             # es el comando de traducción, -s es para comprimir y seguidamente lo que se
20             # quiere comprimir (en este caso espacios)
21             linea_limpia=$(echo "$linea" | tr -s ' ')
22
23             # Extraemos los permisos (primer campo)
24             # cut -d' ' -f1, para cada línea de entrada, la divide en campos usando espacios
25             # como separadores, y muestra solo el primer campo.
26             permisos=$(echo "$linea_limpia" | cut -d' ' -f1)
27
28             # Extraemos el tamaño (quinto campo)
29             tam=$(echo "$linea_limpia" | cut -d' ' -f5)
30             #Extraemos el nombre (a partir del campo 9)
31             nombre=$(echo "$linea_limpia" | cut -d' ' -f9-)

```

```

29
30     #Se necesitan los caracteres 2,3,4 (pertenecientes al propietario)
31     #${variable:posicion:longitud}, se empieza en 1 y se toman 3 caracteres
32     permProp=${permisos:1:3}
33
34     # Guardamos en el temporal: "permisos tamaño nombre"
35     echo "$permProp $tam $nombre" >> "$temp"
36     fi
37     done < "$temp.ls"
38
39     echo ""
40     echo "Agrupación por permisos"
41
42     #Obtenemos los permisos distintos
43     #cut -d" " -f1 extrae el primer campo (permisos)
44     #sort ordena alfabéticamente y uniq elimina duplicados
45     for permiso in $(cut -d" " -f1 "$temp" | sort | uniq); do
46         echo ""
47         echo "Permisos del propietario: $permiso"
48
49         #Mostramos solo los que tengan ese permiso y se ordenan por tamaño
50         #grep "^permiso " Busca líneas que empiecen con el valor de $permiso seguido de un
51         #espacio (^ indica que es al principio de cada línea)
52         #sort -k2 -n ordena numéricamente por el segundo campo (tamaño)
53         grep "^$permiso " "$temp" | sort -k2 -n | while read -r perm tam nom; do
54             printf " Tamaño: %6d bytes -> %s\n" "$tam" "$nom"
55         done
56     done
57
58     #Eliminamos el archivo temporal
59     rm -f "$temp"
60     echo ""
61 }

```

Esta función usa `ls -l` para obtener permisos y tamaños, comprime espacios con `tr -s ' '` y extrae campos con `cut`. La selección de permisos del propietario se obtiene con `${permisos:1:3}` y se agrupa por esos permisos. Cada grupo se ordena por tamaño con `sort -k2 -n` y se filtra con `grep "^$permiso "`, que busca líneas que empiezan por el permiso del propietario.

El formato largo de `ls -l` incluye permisos, enlaces, propietario, grupo, tamaño, fecha y nombre. Como los espacios no son uniformes, `tr -s ' '` convierte múltiples espacios en uno solo, haciendo fiable el uso de `cut -d' ' -fN`. Se extraen permisos (f1), tamaño (f5) y nombre (f9-). Los permisos del propietario son los caracteres 2-4 del campo de permisos, por eso se usa `${permisos:1:3}`. Luego se recorren los permisos distintos con `sort | uniq`, se filtran con `grep "^$permiso "` y se ordenan por tamaño con `sort -k2 -n`. El `while read` final formatea la salida por grupos.

6) Función `ordenar_mes`

```

1 #Función -e, ordenar por inode y agrupar por mes de último acceso
2 ordenar_mes() {
3     echo "Ordenando por inode y agrupado por mes de último acceso"
4

```



```

5 temp=$(mktemp)
6
7 # ls -li --time=atime muestra
8 # -l formato largo
9 # -i muestra el número de inode
10 # --time=atime usa fecha de último acceso
11
12 ls -li --time=atime > "$temp.ls"
13
14 #Leemos línea a línea
15 while read linea; do
16
17     # Evitamos línea "total"
18     if [[ "$linea" != total* ]]; then
19         # Limpiamos espacios múltiples
20         linea_limpia=$(echo "$linea" | tr -s ' ')
21
22         # Extraemos inodo (primer campo)
23         inodo=$(echo "$linea_limpia" | cut -d' ' -f1)
24
25         # Extraemos el mes (campo 7)
26         mes=$(echo "$linea_limpia" | cut -d' ' -f7)
27
28         #Extraemos el nombre (campo 10)
29         nombre=$(echo "$linea_limpia" | cut -d' ' -f10-)
30         echo "$mes $inodo $nombre" >> "$temp"
31     fi
32 done < "$temp.ls"
33
34 echo ""
35 echo "Agrupación por mes de último acceso:"
36
37 # Recorremos todos los meses posibles en orden cronológico (en español como aparecen
38 # al hacer ls -li --time=atime)
39 for mes in ene feb mar abr may jun jul ago sep oct nov dic; do
40
41     #Verificamos que haya archivos en este mes, -c cuenta líneas que coinciden
42     numArch=$(grep -c "^$mes " "$temp")
43     if [ $numArch -gt 0 ]; then
44         echo ""
45         echo "Mes: $mes ($numArch archivos)"
46
47         # Mostramos solo los de ese mes
48         # grep "^$mes " "$temp" busca en el archivo temporal $ todas las líneas que
49         # empiezan con el mes actual del bucle
50         #sort -k2 -n ordena las líneas por el segundo campo (inodo) numéricamente
51         grep "^$mes " "$temp" | sort -k2 -n | while read -r mes inodo nombre; do
52             printf " inode: %s -> %s\n" "$inodo" "$nombre"
53         done
54     fi
55 done
56
57 rm -f "$temp"
58 echo ""
59 }

```

Aquí se usa `ls -li -time=atime` para incluir inode y fecha de último acceso. Con `cut` se

extraen los campos y se agrupa por mes, ordenando por inode dentro de cada mes. El bucle recorre los meses en orden cronológico para mantener una salida consistente.

La opción `-l` genera salida detallada y `-i` añade el inode; con `-time=atime` se usa la fecha del último acceso. Tras normalizar espacios con `tr -s ' '`, el mes se obtiene del campo 7 y el nombre del campo 10 en adelante. El bucle de meses filtra con `grep -c` para saber si hay entradas en ese mes y, si las hay, las ordena por inode con `sort -k2 -n`. Esto produce bloques ordenados por mes.

7) Selección final con **case**

```
1 #Usamos un case para manejar las distintas opciones
2 case $opcion in
3   -a) ordenar_caracteres ;;
4   -b) ordenar_nombreInv ;;
5   -c) ordenar_inode ;;
6   -d) ordenar_permisos ;;
7   -e) ordenar_mes ;;
8   *) #cualquier otra opción que no sea las anteriores
9     echo "Opción inválida"
10    echo "Uso: $0 directorio [-a|-b|-c|-d|-e]"
11    exit 1
12    ;;
13 esac
```

El `case` dirige el flujo a la función adecuada según `$opcion`. Cada rama termina con `;;` y el comodín `*` captura opciones no válidas, mostrando el uso correcto y saliendo con error.

Cada etiqueta `-a`, `-b`, `-c`, `-d` o `-e` llama a la función correspondiente. Si la opción no coincide, el caso comodín `*` actúa como “cualquier otra cosa” y muestra un mensaje de uso antes de finalizar. Esto evita que el script continúe con una opción no soportada.